

Lançar a V1 de teste — uma empresa, gratuito, sem Docker

Este documento responde a três perguntas: **está pronto?**, **onde é que se põe a correr sem pagar nem instalar Docker?** e **como se configura lá?**

Verificado contra o código em **16 de Agosto de 2026**. Os números aqui são medidos, não estimados.

1. Está pronto para uma empresa a sério?

Sim, para uma empresa em avaliação. Com três reservas escritas no ponto 6.

O que foi verificado agora, e não «da última vez»:

Verificação	Resultado
Testes do backend (<code>pytest</code>)	473 passam , 0 falham, 2 min
Build de produção do frontend (<code>npm run build</code>)	passa , sem erros de tipos
Migrações da base (<code>alembic upgrade head</code>)	aplicadas até <code>a7f3c19d84b2</code>
Recusa de arranque com definições de desenvolvimento	activa e testada
Módulos migrados do Piloto	Contabilidade, Analítica, Contas Correntes, Logística, Comercial, Imobilizados, RH, Fiscalidade

E as defesas que já não dependem de ninguém se lembrar: com `AMBIENTE=producao` o backend **recusa arrancar** se o CORS apontar para `localhost` ou `http://`, se faltar a `TOTP_CHAVE_CIFRA`, se a base for a local, ou se a política de palavras-passe estiver abaixo do mínimo. A mensagem diz qual das quatro é.

2. Onde pôr isto a correr, de graça e sem Docker

O Docker não faz falta nenhuma aqui: os alojamentos abaixo constroem a partir do `requirements.txt` e do `package.json` com os seus próprios runtimes. A virtualização desligada no Windows não impede nada disto.

Peça	Onde	Plano	O que custa em incómodo
Backend (FastAPI)	Render	gratuito	Adormece com 15 min sem pedidos; o primeiro pedido a seguir demora cerca de 1 minuto . 750 horas/mês por conta.
Frontend (Next.js)	Render, o mesmo serviço	gratuito	O mesmo adormecer.
Base de dados	Neon	gratuito	0,5 GB e 100 horas de compute/mês. Adormece em 5 min e acorda em menos de 1 s. Permanente , não é um teste de 30 dias.

Porquê o Neon e não a Postgres do Render: a base gratuita do Render **expira 30 dias depois de criada** e é apagada 14 dias depois. Um piloto numa empresa dura mais do que isso, e perder os lançamentos a meio da avaliação estragava a avaliação.

Porquê o Render e não a Vercel para o frontend: a Vercel é, tecnicamente, a melhor casa para Next.js — mas o plano gratuito (Hobby) é, nas palavras deles, «*non-commercial, personal use only*». Pôr lá o ERP de uma empresa a trabalhar é uso comercial. Fica a escolha explícita: **Render** para não haver dúvida, ou **Vercel Pro (20 USD/mês)** se preferir a experiência de Next.js deles.

Quando isto deixar de chegar (V2, várias empresas): 7 USD/mês por serviço no Render tiram o adormecer, e o Neon tem escalões pagos a partir de 5 USD. Fica na casa dos 20 USD/mês — e aí já se justifica um domínio próprio.

3. Passo a passo

Do zero à primeira entrada. Precisa de: uma conta no GitHub com este repositório, uma conta no Render, uma no Neon, e o Python que já tem na máquina.

3.1 A base de dados (Neon)

1. Criar conta em neon.tech → **New Project**, região **Frankfurt** (a mais perto).

1. Copiar a **Connection string**. Tem esta forma:

```
postgresql://utilizador:palavra@ep-xxx.eu-central-1.aws.neon.tech/neondb?sslmode=require
```

1. Guardar. É a `DATABASE_URL`.

Não é preciso mudar nada na linha: a aplicação reconhece a forma `postgresql://` e mete-lhe o condutor certo sozinho.

3.2 Criar as tabelas e a primeira conta — da SUA máquina

O plano gratuito do Render **não dá acesso a consola**. Por isso estes dois passos correm daqui, contra o Neon, que está aberto à Internet com TLS:

```
cd Producao/backend
```

Pôr a ligação do Neon no ambiente e migrar:

```
DATABASE_URL="postgresql://...neon.tech/neondb?sslmode=require" ./venv/Scripts/python.exe -m alembic upgrade head
```

Criar a conta de administração da plataforma (pede nome, e-mail e palavra-passe; não aceita nada por argumento, para não ficar no histórico da shell):

```
DATABASE_URL="postgresql://...neon.tech/neondb?sslmode=require" ./venv/Scripts/python.exe  
scripts/criar_superadmin.py
```

Não corra o `criar_demo.py`. Ele recusa-se a correr em produção, e é bem que recuse: são contas com palavras-passe conhecidas.

3.3 Os dois serviços (Render)

1. Render → **New** → **Blueprint** → escolher este repositório.

Ele lê o `render.yaml` da raiz e propõe **dois** serviços: `aycontabilidade-api` e `aycontabilidade-web`.

1. Vai pedir as variáveis que não estão no ficheiro. Preencher:

No **api**:

- `DATABASE_URL` → a linha do Neon;
- `CORS_ORIGINS` → fica em branco por agora (ainda não se sabe o endereço

do frontend). **O serviço vai falhar a arrancar** — é suposto, e a mensagem di-lo. Corrige-se no passo 4.

- `OPENAI_API_KEY` → a chave, ou vazio. Sem ela o assistente fica

indisponível e **todo o resto funciona**, incluindo o diagnóstico, que é local.

No **web**:

- `NEXT_PUBLIC_API_URL` → também só se sabe a seguir; pôr

`https://aycontabilidade-api.onrender.com` (é o endereço que o Render costuma dar; confirma-se no passo 3).

1. Deixar construir. Anotar os dois endereços que o Render atribui:

`https://aycontabilidade-api.onrender.com` e `https://aycontabilidade-web.onrender.com`.

1. Fechar o círculo, que é onde toda a gente se engana:

- no **api**, `CORS_ORIGINS` = o endereço **do web**, em `https`, ****sem**

barra no fim**;

- no **web**, `NEXT_PUBLIC_API_URL` = o endereço **do api**;
- **Redeploy** aos dois. O `NEXT_PUBLIC_` entra no build: mudá-lo obriga a

construir outra vez, não basta reiniciar.

3.4 Primeira entrada

1. Abrir `https://aycontabilidade-web.onrender.com` — a primeira abertura demora até um minuto (os serviços estavam a dormir).

1. Entrar com a conta do passo 3.2, **deixando o campo da empresa vazio** — é uma conta de plataforma.

1. **Activar o segundo factor.** A área de administração exige-o e fica

fechada até lá.

1. Criar a licença e a empresa: **Plataforma** → **Licenças**.
2. Criar o administrador da empresa: **Plataforma** → **Contas**.
3. Entregar a essa pessoa o endereço, o código da empresa e a palavra-passe temporária. Ela muda-a na primeira entrada.

4. Configurar depois de lançado

Tudo o que muda o comportamento está em variáveis de ambiente, no painel do serviço (**Environment**). Guardar reinicia o serviço.

Variável	Para quê	Cuidado
<code>CORS_ORIGINS</code>	quem pode falar com a API	vários endereços separados por vírgula; <code>https</code> obrigatório
<code>ACCESS_TOKEN_MINUTOS</code>	duração da sessão inactiva	30 é o valor de fábrica
<code>SESSAO_ABSOLUTA_HORAS</code>	tecto absoluto da sessão	12 — mesmo com uso contínuo, obriga a entrar de novo
<code>PASSWORD_MIN_CARACTERES</code>	política de palavras-passe	mínimo 8; abaixo disso não arranca
<code>RATE_LIMIT_LOGIN</code>	tentativas de entrada	<code>5/minute</code>
<code>OPENAI_MODELO</code>	modelo do assistente	ver <code>GET /api/ia/modelos</code> para saber o que a chave alcança
<code>AGT_ATIVO</code>	consulta de NIF à AGT	precisa de credenciais da AGT

A `TOTP_CHAVE_CIFRA` **não se muda depois de haver contas com segundo factor activo**. Mudá-la torna ilegíveis os segredos guardados e obriga toda a gente a reconfigurar.

5. Cópias de segurança — isto é consigo

Nem o Render gratuito nem o Neon gratuito garantem cópias. Numa avaliação com dados de uma empresa real, uma cópia por semana é o mínimo:

```
pg_dump "postgresql://...neon.tech/neondb?sslmode=require" -Fc -f copia-2026-08-16.dump
```

Guardar fora do computador de trabalho. Restaurar é `pg_restore`.

6. As três reservas

Escritas por inteiro para a decisão ser informada, e não descobertas a meio do piloto.

1. Facturação com certificação da AGT — por decidir. O sistema emite facturas e mapas, mas a certificação de software de facturação junto da AGT é um processo administrativo que ainda não foi decidido nem iniciado. Uma empresa angolana não pode emitir facturas legais com software não certificado. Para avaliar contabilidade, RH, existências e relatórios, isto não estorva; para facturar a clientes, estorva. Ver `docs/documentacao/PENDENCIAS_PRIORITARIAS.md`.

2. O adormecer é visível. Um minuto de espera na primeira utilização do dia faz uma primeira impressão pior do que o produto merece. Se a avaliação for levada a sério por quem lá trabalha, os 7 USD/mês do escalão pago do serviço do backend são o dinheiro mais bem gasto do projecto.

3. Coluna do exercício anterior no Balanço. Fica vazia enquanto a empresa não tiver dois exercícios fechados. É o comportamento correcto — não há de onde tirar o número —, mas só se confirma quando existir o segundo exercício.

7. Do teste (V1) para várias empresas (V2)

O que a V1 tem de responder antes de haver V2:

- os lançamentos do dia-a-dia fecham o mês sem intervenção manual?
- os mapas batem certo com o que a contabilidade da empresa já produzia?
- o processamento de salários dá o mesmo que o método actual, ao cêntimo?
- alguém pediu alguma coisa que não existe?

A arquitectura já é multiempresa desde o primeiro dia — licença, módulos e capacidades por conta —, por isso a V2 não é uma reescrita: é mudar de escalão, pôr um domínio próprio e repetir o ponto 3 uma vez só.
